

PROJECT REPORT

On

EPL MatchVision

(ML Based EPL Football Match Outcome & Score Prediction)

(A Web-Based Application for Predicting Full-time Scores & Win Probabilities Using Machine Learning)

A dissertation submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering of Maulana Abul Kalam Azad University of Technology, West Bengal



Submitted by

Sawmik Pal(11200122033)
Sayar Sekhar Ghosh(11200122029)
Sarthak Mukherjee(11200123073)
Pramiti Ghosh(11200122060)
Soumik Mandal(11200122053)

Under the Guidance of

Dr. Debayan Ganguly

Assistant Professor, Department of Computer Science & Engineering

Government College of Engineering and Leather Technology

(Affiliated to MAKAUT, West Bengal)

Kolkata - 700106, West Bengal

2025-2026

CERTIFICATE

This is to certify that the Project Report titled "**EPL MatchVision – From Historical Data to Matchday Predictions**" submitted by the following students is an original work carried out under academic guidance in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Government College of Engineering and Leather Technology, affiliated to Maulana Abul Kalam Azad University of Technology, West Bengal.

Name of Student	Roll No.	Signature
SAWMIK PAL	11200122033	
SAYAR SEKHAR GHOSH	11200122029	
SARTHAK MUKHERJEE	11200123073	
PRAMITI GHOSH	11200122060	
SOUMIK MANDAL	11200122053	

The project has been completed satisfactorily and the report has been prepared according to the prescribed academic format. To the best of our knowledge, the work has not been submitted for any other degree, diploma, or academic award.

Date: _____ Place: Kolkata

Project Guide	Head of the Department / Departmental Approval
Signature: _____ Dr. Debayan Ganguly Assistant Professor, Department of Computer Science & Engineering Government College of Engineering and Leather Technology	Signature: _____ Dr. Brijesh Shrivastava Head of the Department, Department of Computer Science & Engineering Government College of Engineering and Leather Technology

STUDENT DECLARATION

We hereby declare that the Project Report titled "**EPL MatchVision –From Historical Data to Matchday Predictions** " submitted to Maulana Abul Kalam Azad University of Technology, West Bengal in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science & Engineering is our original project work.

The design, implementation, testing observations, analytical interpretation, and conclusions presented in this report are based on the project developed by our team. The work has not been submitted elsewhere for any degree, diploma, or academic award.

Date: _____ Place: Kolkata

Name of Student	Roll No.	Signature
SAWMIK PAL	11200122033	
SAYAR SEKHAR GHOSH	11200122029	
SARTHAK MUKHERJEE	11200123073	
PRAMITI GHOSH	11200122060	
SOUMIK MANDAL	11200122053	

NO-PLAGIARISM DECLARATION

Title of the Project: **EPL MatchVision – From Historical Data to Matchday Predictions**

Names of Learners: SAWMIK PAL, SAYAR SEKHAR GHOSH, SARTHAK MUKHERJEE, PRAMITI GHOSH, SOUMIK MANDAL

Project Supervisor: **Dr. Debayan Ganguly**

We declare that this Project Report is the result of our independent project work. All sources of information, documentation, tools, libraries, frameworks, and external references used for understanding and development have been duly acknowledged in the References section.

No portion of this work has been copied or reproduced from another project report or publication without proper acknowledgement. We understand that plagiarism is a serious academic offence and may result in rejection of the work and appropriate academic action.

Date: _____ Place: Kolkata

Name of Student	Roll No.	Signature
SAWMIK PAL	11200122033	
SAYAR SEKHAR GHOSH	11200122029	
SARTHAK MUKHERJEE	11200123073	
PRAMITI GHOSH	11200122060	
SOUMIK MANDAL	11200122053	

ACKNOWLEDGEMENT

With immense pleasure and a deep sense of gratitude, we express our sincere thanks to **Dr. Brijesh Shrivastava**, Head of the Department of Computer Science & Engineering, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project.

We are extremely thankful to **Dr. Debayan Ganguly**, Assistant Professor, Department of Computer Science & Engineering, for his academic support, patience, motivation, and guidance, which helped us to design and implement our project, "**EPL MatchVision – From Historical Data to Matchday Predictions**".

We also express our gratitude to all faculty members of the Department of Computer Science & Engineering for their cooperation, technical suggestions, and academic support during the development phase. Their encouragement helped us improve the system design, documentation quality, and presentation of the project.

Finally, we are deeply grateful to our parents and friends for their continuous support, motivation, and understanding, without which this project would not have been possible.

Submitted by:

Name of Student	Roll No.
SAWMIK PAL	11200122033
SAYAR SEKHAR GHOSH	11200122029
SARTHAK MUKHERJEE	11200123073
PRAMITI GHOSH	11200122060
SOUMIK MANDAL	11200122053

ABSTRACT

Accurately predicting professional football match outcomes remains a complex analytical challenge due to the dynamic nature of the sport. Processing historical data, rolling team form, and custom performance metrics is mathematically intensive and difficult to interpret manually. This project presents EPL MatchVision, a data-driven web application that enables users to dynamically select competing English Premier League teams, evaluate machine learning algorithms, and retrieve full-time score predictions, win probabilities, and expected goals.

EPL MatchVision integrates a Streamlit frontend with a modern, high-contrast football field interface, custom CSS styling, responsive layout containers, and dynamic official club logos mapped from open-source repositories. The predictive pipeline is developed using Python, Scikit-Learn, and XGBoost, processing historical English Premier League match data from 2000 to 2025. The core analytical novelty of this project lies in its comparative feature engineering approach. The application evaluates four distinct machine learning models. The Basic Random Forest model is trained utilizing rolling average statistics to capture a team's recent form. To overcome the limitations of short-term form tracking, the project introduces a custom Elo rating system as a novel, long-term performance metric. The remaining three models—Random Forest with Elo Ratings, XGBoost Classifier and Regressor, and a Multi-Layer Perceptron (MLP) Neural Network—are trained on this newly introduced Elo rating framework. These models return a structured classification of discrete match outcomes and a regression of expected goals.

The system utilizes joblib serialization to separate the computationally heavy model training phase, conducted in Jupyter Notebooks, from the web interface deployment. This architecture avoids repeated model training during frontend interaction and enables cached inference from pre-trained artifacts. In the final stage, EPL MatchVision was extended with important structural and usability improvements: dedicated routing for exploring the raw dataset, tracking team Elo ratings, reviewing model accuracy metrics, and visual probability progress bars.

The final evaluation showed that XGBoost achieved the highest classification accuracy (54.58%) and the lowest average home/away goal MAE (0.939 goals). Adding Elo features to Random Forest improved classification accuracy from 48.98% to 54.17%, demonstrating the value of long-term team-strength information. The system is therefore presented as an analytical decision-support prototype rather than a deterministic match-forecasting engine.

CONTENTS

Sr. No.	Content	Page No.
1	Certificate	2
2	Student Declaration	3
3	No-Plagiarism Declaration	4
4	Acknowledgement	5
5	Abstract	6
6	List of Illustrations	8
7	Chapter 1: Introduction	9
8	Chapter 2: Objectives of the Study	12
9	Chapter 3: Literature Review	14
10	Chapter 4: Problem Statement	17
11	Chapter 5: Methodology	19
12	Chapter 6: Data Analysis and Interpretation	22
13	Chapter 7: System Design	31
14	Chapter 8: Testing, Discussion and Results	38
15	Chapter 9: Conclusion	45
16	Chapter 10: Recommendations and Future Scope	47
17	References	48
18	Appendix A: Evaluation Notes	49

LIST OF ILLUSTRATIONS

Illustration No.	Title	Page No.
Table 2.1	Sub-objectives mapped with project modules	13
Table 4.1	Problem decomposition and project response	18
Table 6.1	Random Forest confusion matrix	24
Table 6.2	Random Forest precision, recall and F1-score	24
Table 6.3	Random Forest with Elo confusion matrix	25
Table 6.4	Random Forest with Elo class-wise metrics	26
Table 6.5	XGBoost confusion matrix	27
Table 6.6	XGBoost class-wise metrics	27
Table 6.7	MLP confusion matrix	28
Table 6.8	MLP class-wise metrics	29
Table 6.9	Functional observation matrix	30
Figure 7.1	Overall architecture of EPL MatchVision	32
Figure 7.2	Level 0 context diagram	33
Figure 7.3	Level 1 data-flow diagram	34
Figure 7.4	Use-case model	35
Figure 7.5	Prediction-request sequence	36
Figure 7.6	Data serialization and storage design	37
Table 8.1	Comparative model evaluation summary	38
Figure 8.1	Classification accuracy comparison	39
Figure 8.2	Average goal MAE comparison	39
Figure 8.3	Home Page with Navigation Bar	41
Figure 8.4	Model performance dashboard	41
Figure 8.5	Team Elo ratings dashboard	41
Figure 8.6	Developer Information	42
Figure 8.7	Information About EPL and Teams	42
Figure 8.8	Prediction dashboard showing probabilities and expected scoreline	42
Table 8.2	Objective fulfilment and final outcomes	43

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

In the modern sports domain, football analytics has rapidly evolved from intuition-based predictions to highly sophisticated, data-driven forecasting. Fans, analysts, and sports management now rely heavily on computational models to process historical match records, track team form, and evaluate complex performance metrics. However, processing this vast amount of statistical data manually is mathematically intensive and prone to human bias. Furthermore, standard predictive approaches often struggle to balance short-term team form with long-term historical strength. Supervised machine learning provides a robust solution to this challenge, allowing systems to learn mathematical mappings from historical statistics to predict both discrete match outcomes and continuous variables like expected goals.

1.2 Project Overview

EPL MatchVision is a comprehensive, interactive web application designed to forecast English Premier League (EPL) match outcomes using Machine Learning. The application acts as an analytical dashboard where users can dynamically select competing teams, choose a specific predictive algorithm, and instantly retrieve full-time score predictions, win/draw/loss probabilities, and expected goals.

The system is formulated strictly as a Supervised Machine Learning pipeline. It evaluates and compares four distinct predictive models: Basic Random Forest, Random Forest integrated with a custom Elo Rating system, XGBoost Classifier and Regressor, and a Multi-Layer Perceptron (MLP) Neural Network. [3-5]

The frontend interface is built using Streamlit, featuring a modern, high-contrast football field interface layout. It includes custom CSS styling, dynamic responsive containers, and official club logo integration fetched from open-source repositories. To ensure high performance, the system separates the computationally heavy model training phase from the web interface using joblib serialization, which allows the application to load pre-trained models and state dictionaries instantly without redundant training delays.

1.3 Rationale of the Study

Accurately predicting professional football match outcomes remains a complex analytical challenge due to the dynamic nature of the sport. The primary rationale behind this project is to overcome the limitations of standard short-term statistical tracking. While basic models rely merely on rolling

averages of recent matches, this project introduces a custom Elo rating system to serve as a novel, long-term performance metric that accurately reflects a team's enduring historical strength.

Additionally, a significant technical rationale was addressing frontend performance bottlenecks. Integrating heavy machine learning workflows directly into web scripts typically causes massive execution delays during user interactions. By designing a decoupled architecture—where models are trained in Jupyter Notebooks and serialized as binary files to be executed by a lightweight Streamlit web app—this project demonstrates an industry-standard method for deploying machine learning applications efficiently.

1.4 Scope of the Project

- **Data Processing:** Ingesting and engineering features from a comprehensive Kaggle dataset of historical EPL matches spanning from 2000 to 2025.
- **Predictive Modeling:** Utilizing Scikit-Learn and XGBoost to deploy both Classification (predicting Home Win, Draw, or Away Win) and Regression (predicting exact expected goals).
- **Feature Engineering:** Tracking dynamic team form via rolling averages (e.g., goals scored, goals conceded) and implementing a custom Elo Rating framework for robust historical tracking.
- **User Interface:** Providing a responsive, multi-page web dashboard that includes visual probability progress bars, dynamic team logos, and dedicated routing for exploring the raw dataset and reviewing model accuracy metrics.
- **Performance Optimization:** Utilizing state serialization (joblib) to reduce startup overhead and execute predictions without retraining the models.

1.5 Limitations of the Current Scope

- **Data Constraints:** The predictions are based entirely on historical numerical data and structured rolling statistics. The models cannot account for sudden, unquantifiable real-world variables such as

unexpected player injuries, weather conditions, or sudden managerial changes.

- **Pre-Match Focus:** The system is designed strictly for pre-match forecasting. It does not ingest live match data or provide dynamically updating in-play predictions.
- **Platform Dependency:** The application requires a stable environment to execute Python and load large serialized binary model files, and currently relies on external open-source repositories to fetch team logo assets.

CHAPTER 2: OBJECTIVES OF THE STUDY

2.1 Main Objective

The main objective of EPL MatchVision is to design and develop a high-performance, data-driven web application that evaluates multiple machine learning algorithms to accurately predict English Premier League (EPL) full-time scores, win probabilities, and expected goals by synthesizing historical match records and custom team performance metrics.

2.2 Specific Objectives

1. To build a responsive, interactive frontend interface using Streamlit, featuring a premium, high-contrast football field interface layout and integrated with dynamic official club logos.
2. To implement and compare a robust predictive pipeline evaluating four distinct machine learning models: Basic Random Forest, Random Forest integrated with Elo Ratings, XGBoost Classifier and Regressor, and a Multi-Layer Perceptron (MLP) Neural Network.
3. To engineer advanced predictive features that capture both a team's short-term form through rolling averages (e.g., goals scored, conceded, and points over the last five matches) and long-term historical strength through the introduction of a custom Elo rating system.
4. To optimize system architecture by strictly separating the computationally intensive model training phase (conducted in Jupyter Notebooks) from the web deployment using joblib serialization, thereby eliminating repeated training during frontend interactions and reducing unnecessary computation.
5. To design dedicated routing modules within the dashboard that allow users to explore raw historical datasets (2000-2025), review comparative model accuracy metrics, and track dynamically updating team Elo ratings.

2.3 Sub-Objectives Mapped with Project Modules

To systematically solve the primary problem statement, the core objectives have been broken down and mapped to specific technical modules.

Table 2.1: Sub-Objectives Mapped with Project Modules

Objective Area	Corresponding Module	Expected Outcome
User Interface & UX	Streamlit Web Dashboard	Users can seamlessly select competing teams and predictive algorithms in a modern, cyber-themed environment with visual probability progress bars.
Feature Engineering	Data Utility & Elo Tracker	The machine learning models receive robust, structured features that accurately reflect both short-term rolling match form and long-term historical team strength.
Predictive Modeling	ML Pipeline (Classification & Regression)	The system generates accurate discrete match outcomes (Home Win, Draw, Away Win) alongside exact expected goal predictions for both teams.
Performance Optimization	joblib Serialization Layer	The frontend bypasses redundant training execution by loading cached model resources once and reusing them for subsequent inference requests.
Data Exploration	Secondary Routing Pages	Users can analytically verify the system by exploring the underlying Kaggle dataset, current Elo ratings, and individual algorithm performance metrics.

CHAPTER 3: Literature Review

3.1 Introduction

Football has become one of the most data-driven sports in the world. With the rapid growth of machine learning, statistical modelling, and sports analytics, researchers have attempted to predict football match outcomes using historical match data, player statistics, betting odds, team ratings, and advanced performance metrics. Predicting football matches is a challenging problem because the game is influenced by numerous dynamic factors including team form, player injuries, tactical decisions, weather conditions, home advantage, and psychological aspects.

Recent advances in machine learning have significantly improved prediction accuracy by combining historical data with engineered features such as Elo ratings, rolling averages, expected goals (xG), and team momentum. This chapter reviews the major research works related to football prediction systems and identifies the research gap addressed by the proposed system.

3.2 Machine Learning in Football Prediction

Machine learning techniques have become increasingly popular for predicting football matches because they can learn complex nonlinear relationships between numerous match variables.

Traditional statistical models such as Logistic Regression and Poisson Regression were initially used for predicting match outcomes. However, modern ensemble algorithms such as Random Forest, Gradient Boosting, XGBoost and Neural Networks generally achieve better predictive performance due to their ability to model nonlinear interactions among features.

Most football prediction systems formulate the problem either as:

- 1. Classification**

- a. Home Win
- b. Draw
- c. Away Win

Or,

- 2. Regression**

- a. Home Goals
- b. Away Goals

Some modern systems combine both tasks to simultaneously predict match results and expected scores.

3.3 Previous Research Works

Study 1: Baboota and Kaur (2018)

Link:

https://www.researchgate.net/publication/324072605_Predictive_analysis_and_modelling_football_results_using_machine_learning_approach_for_English_Premier_League

Baboota and Kaur developed an English Premier League prediction system using Gradient Boosting algorithms.

The authors compared several machine learning algorithms including:

- Logistic Regression
- Support Vector Machine
- Random Forest
- Gradient Boosting

The Gradient Boosting model achieved the best performance among all tested models.

Contribution:

- Demonstrated superiority of boosting methods
- Introduced extensive feature engineering
- Compared predictions against bookmaker odds

Limitation:

- Dataset covered only a few seasons.
- Real-time deployment was not discussed.

Study 2: Bunker et al. (2024)

Link:

<https://doi.org/10.48550/arXiv.2403.07669>

Bunker et al. presented a comprehensive survey on machine learning approaches for football prediction.

The study reviewed:

- Decision Trees
- Random Forest
- CatBoost

- XGBoost
- Deep Neural Networks
- LSTM
- Graph Neural Networks

The authors concluded that Gradient Boosting methods currently provide the best balance between prediction accuracy and computational efficiency.

Contribution:

- Comprehensive survey
- Comparison of modern ML algorithms
- Identification of future research directions

Limitation:

- Focused mainly on research comparison rather than implementation.

Study 3: Wong et al. (2025)

Link:

<https://doi.org/10.1016/j.dajour.2024.100537>

Wong et al. proposed a predictive analytics framework using ensemble machine learning methods.

The framework incorporated:

- Weather conditions
- Team fatigue
- Momentum
- Historical statistics

The study demonstrated that incorporating external contextual information improved match prediction accuracy.

Contribution:

- Multi-feature ensemble framework
- Weather integration
- Momentum analysis

Limitation:

- Computational complexity increased significantly.

CHAPTER 4: PROBLEM STATEMENT

4.1 Existing Problem

In the domain of football analytics, users and analysts frequently face the challenge of processing vast amounts of historical match data to predict future outcomes. Manually calculating complex variables—such as rolling goal averages, team form, and historical strength—is mathematically intensive and prone to human bias. Furthermore, a significant technical problem exists in deploying machine learning models for public use. When integrating heavy predictive workflows into interactive web applications like Streamlit, the entire Python script re-runs from top to bottom whenever a user interacts with a widget. If the model training and feature extraction code remains coupled with the frontend script, it forces the system to redundantly retrain models on every click, causing massive execution delays, freezing interfaces, and rendering the application practically unusable for real-time forecasting.

4.2 Research Gap Identified

The core research gap addressed by this project is twofold. First, from an analytical perspective, standard predictive models often rely entirely on short-term statistical tracking (e.g., the last five matches), failing to account for a team's enduring historical strength. Second, from a software engineering perspective, there is a lack of accessible, optimized dashboards that correctly decouple machine learning training phases from web deployment. Many existing approaches fail to serialize complex state data, meaning functions that calculate rolling averages or custom tracking dictionaries are unnecessarily recomputed on the frontend. This project bridges both gaps by introducing a custom Elo rating system as a long-term performance metric and utilizing an optimized, decoupled system architecture.

4.3 Formal Problem Statement

To design and develop a high-performance, machine-learning-powered web application that accurately predicts English Premier League match outcomes, expected goals, and win probabilities by synthesizing rolling match statistics and custom Elo ratings, while overcoming frontend performance bottlenecks through strict state serialization and decoupled model deployment.

4.4 Problem Decomposition

To systematically address the formal problem statement, the primary challenges have been broken down into distinct technical components and mapped to specific project responses, as detailed in the table below.

Table 4.1: Problem decomposition and project response

Problem	Component Challenge	Project Response
Statistical Complexity	Users cannot easily or accurately compute multi-variable probabilities manually.	Automated Machine Learning pipeline evaluating four algorithms (Basic Random Forest, Elo RF, XGBoost, and MLP Neural Network) to generate instant predictions.
Frontend Performance Lag	Interactive Streamlit widgets can re-run the script after user input; keeping model training in that path would create substantial delays.	Strict separation of the Training phase from the Inference phase. Models are pre-trained in Jupyter Notebooks and serialized using the joblib library to ensure instant frontend loading.
State Recomputation	Re-calculating complex historical state dependencies (like custom Elo tracking) slows down the prediction output.	Pre-computing and saving the final state dictionaries (e.g., team_history and elo_teams) as binary .joblib files, allowing the web app to execute instant lookups without recalculating past data.
Tracking Gap	Standard feature engineering heavily relies on short-term form, ignoring a team's historical dominance.	Implementation of a custom Elo Rating System designed to track and weigh a team's long-term historical strength across seasons.
UI/UX Limitations	Default data presentation interfaces often look monotonous, generic, and difficult to interpret quickly.	Deployment of a premium, custom-styled football field theme dashboard featuring visual probability progress bars, responsive metrics, and dynamic club logos fetched from open-source repositories.

CHAPTER 5: METHODOLOGY

5.1 Research Design

The project utilizes an applied, quantitative research design formulated strictly as a Supervised Machine Learning pipeline. Rather than relying on reinforcement learning or simulation agents, the system leverages historical, labeled data to learn mathematical mappings between past team statistics and future match outcomes. The analytical design is dual-faceted, deploying **Classification** to predict discrete match outcomes (Home Win, Draw, Away Win) and **Regression** to forecast continuous variables, specifically the expected goals for both the home and away teams.

To ensure system efficiency, the architectural design enforces a strict separation of concerns between the training phase and the inference phase. Computationally heavy tasks are isolated in Jupyter Notebooks, while a lightweight web interface handles the user interaction and real-time prediction processing.

5.2 Data Collection Method and Tool

The project relies on secondary historical data. The primary data source is the **"English Premier League (EPL) Match Data 2000-2025"** dataset, which was systematically collected from the Kaggle open-source repository. This dataset provides comprehensive historical match features, including full-time results, goals scored, shots on target, fouls, and card distributions, which act as the foundational inputs for the predictive pipeline. [1]

Kaggle Dataset Link- <https://www.kaggle.com/datasets/marcohuiii/english-premier-league-epl-match-data-2000-2025>

5.3 Sampling Size & Methods

The sample size encompasses 25 years of continuous English Premier League match records (from 2000 to 2025). Because football match data is inherently chronological, random sampling cannot be used, as it would cause data leakage (predicting past matches using future knowledge).

Instead, a strict chronological sampling and partitioning method is employed:

- **Train-Test Split:** The dataset is split temporally. For instance, the models are trained on historical

matches occurring before January 1, 2021, and tested on completely unseen match data occurring on or after January 1, 2021.

- **Cross-Validation:** During hyperparameter tuning, a TimeSeriesSplit (with 5 splits) is utilized to ensure the models are validated progressively across different consecutive football seasons without breaking the temporal sequence. [6]

5.4 Analytical Tools Used

To execute the data processing, predictive modeling, and user interface deployment, the following technical stack and analytical tools are utilized:

- **Python:** The core programming language used across the entire development pipeline.
- **Pandas & NumPy:** Used extensively for data manipulation, cleaning, and structuring the historical match features into mathematical arrays.
- **Scikit-Learn & XGBoost:** The primary machine learning libraries utilized to engineer the four predictive algorithms: Basic Random Forest, Random Forest integrated with Elo Ratings, XGBoost Classifier/Regressor, and the Multi-Layer Perceptron (MLP) Neural Network.
- **Joblib:** A highly efficient serialization library used to save the fully trained machine learning models and dynamic state dictionaries (such as team rolling histories and custom Elo ratings) as binary files. This is the critical tool used to eliminate frontend performance bottlenecks, allowing the web application to load predictions instantly without redundant retraining.
- **Streamlit:** The web framework utilized to deploy the interactive dashboard. The standard Streamlit interface was heavily overridden using custom CSS to provide a premium, modern football field theme UI. [11]

5.5 Development Methodology & Pipeline

The system development methodology follows a four-phase analytical pipeline:

1. **Feature Engineering:** Raw match statistics are transformed into predictive features. This involves tracking a team's short-term form by calculating rolling averages over their last five matches (tracking goals scored, goals conceded, and average points). To capture long-term dominance, a custom Elo Rating system is simultaneously calculated and mapped. [2, 12]
2. **Model Optimization:** Models are optimized using RandomizedSearchCV to find the most accurate

hyperparameters (such as `max_depth` and `n_estimators` for the Random Forest) targeting accuracy for classification and negative mean absolute error scoring during cross-validation for expected goals regression. [7]

3. **State Serialization:** At the end of the training phase, the fully trained algorithms, along with the final historical tracking dictionaries (`team_history` and `elo_teams`), are serialized into `.joblib` files.
4. **Frontend Integration:** The Streamlit dashboard imports these pre-compiled binary models through a caching mechanism (`@st.cache_resource`). The frontend captures user inputs through dropdown menus, executes inference against the serialized historical states on demand, and renders the probabilities through custom metric blocks and progress bars.

CHAPTER 6: DATA ANALYSIS AND INTERPRETATION

6.1 Nature of Data Analyzed

The data analyzed in this project is strictly historical, quantitative, and chronological. The foundational dataset (`epl_final.csv`) contains English Premier League match records encompassing variables such as full-time results, goals scored, shots, fouls, and card distributions. The primary goal of the data analysis phase is to mathematically map these historical statistics to future match outcomes.

For classification purposes, the target variable, `FullTimeResult`, is systematically encoded into discrete numerical identifiers: 0 for Home Win, 1 for Draw, and 2 for Away Win. For regression purposes, the exact full-time home goals and full-time away goals serve as continuous target variables.

6.2 Feature Engineering and Short-Term Form Tracking

Raw statistical totals are insufficient for predictive machine learning because they do not reflect relative team momentum. Therefore, the data was interpreted and engineered into differential and rolling metrics:

- **Match Differentials:** The system calculates immediate match differences, producing engineered features such as `GoalDiff`, `ShotDiff`, `ShotOnTargetDiff`, `FoulDiff`, `YellowDiff`, and `RedDiff`.
- **Rolling Averages (5-Match Form):** To quantify a team's current form, the system analyzes a chronological state dictionary (`team_history`). A custom function (`get_last_n_stats`) iterates through this history to calculate a team's rolling average over their last five matches. This tracks recent wins, draws, losses, average goals scored, average goals conceded, and average points accumulated.

6.3 Long-Term Data Interpretation: The Custom Elo Rating System

While rolling averages successfully interpret a team's short-term form, they fail to capture historical dominance. To bridge this analytical gap, the project implements a custom Elo Rating system. This algorithm adjusts a team's baseline rating dynamically after every match. The final calculated Elo ratings for all 46 historic and current Premier League teams are serialized as an `elo_all.joblib` dictionary. This engineered feature serves as the foundational training metric for the advanced Random Forest, XGBoost, and Multi-Layer Perceptron (MLP) models.

6.3.1 Elo Rating Update Mechanism

The project adapts the Elo paired-comparison principle to football. Before each chronological match, the current ratings of the two teams are read without using future results; after the match, both ratings are updated from the observed outcome [2, 12].

Expected result for Team A: $EA = 1 / (1 + 10^{((RB - RA)/400)})$.

Rating update: $R'A = RA + K(SA - EA)$, where SA is 1 for a win, 0.5 for a draw, and 0 for a loss. The configurable K-factor controls how strongly a single result changes the rating. The final home and away Elo values, together with their difference, are supplied as model features and persisted in `elo_all.joblib`.

6.4 Data Sampling and Cross-Validation Analysis

Because football match data is inherently chronological, standard random train-test splitting would cause "data leakage" (predicting past events using future knowledge).

- **Chronological Split:** The data is strictly partitioned by time. Matches occurring before January 1, 2021, are allocated to the training set (`X_train`, `Y_train`), while matches occurring on or after January 1, 2021, are allocated entirely to the testing set (`X_test`, `Y_test`).
- **Time Series Cross-Validation:** During hyperparameter tuning, the system utilizes `TimeSeriesSplit` (with 5 splits) within a `RandomizedSearchCV` framework. This ensures the models are validated progressively across consecutive football seasons.

6.5 Statistical Evaluation Metrics

The mathematical outputs of the models are interpreted using industry-standard statistical metrics:

- **Classification Evaluation:** The discrete match outcome models are evaluated using `accuracy_score`, `confusion_matrix`, and `classification_report`. This provides a detailed breakdown of precision and recall for Home Wins, Draws, and Away Wins. [8]
- **Regression Evaluation:** The expected-goal models are tuned with the scikit-learn scoring key `neg_mean_absolute_error` during cross-validation. For final testing, the report presents Mean Absolute Error (MAE), where lower values indicate smaller goal-prediction errors [8].

Random Forest with rolling averages:

I. Classification:

a) **Accuracy Score:** 0.4897959183673469

b) **Confusion Matrix:**

	Predicted Home Win	Predicted Draw	Predicted Away Win
Actual Home Win	672	0	81
Actual Draw	307	0	82
Actual Away Win	405	0	168

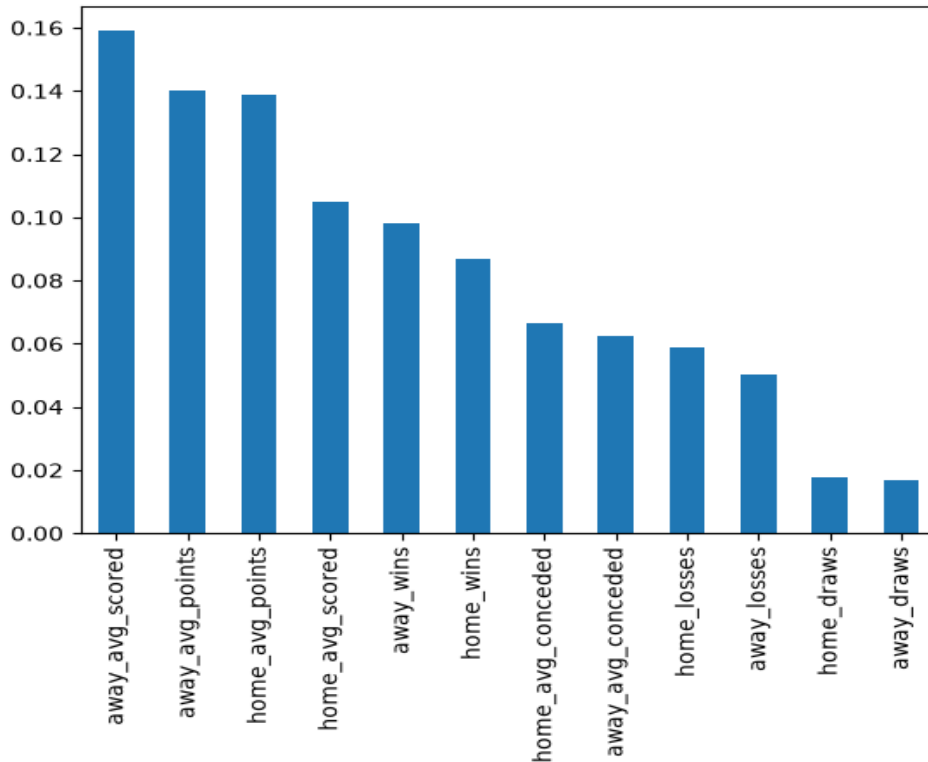
Table 6.1: Random Forest confusion matrix

c) Precision-Recall-F1 score:

	Precision	Recall	F1-Score
Home Win	0.49	0.89	0.63
Draw	0.00	0.00	0.00
Away Win	0.51	0.29	0.37

Table 6.2: Random Forest precision, recall and F1-score

d)Importance of features:



II.Regression:

a) Home Goal MAE: 1.033254569603529

b) Away Goal MAE: 0.9200252317522334

Random Forest with Elo System:

I. Classification:

a)Accuracy Score: 0.5416909620991254

b)Confusion Matrix:

	Predicted Home Win	Predicted Draw	Predicted Away Win
Actual Home Win	648	0	105
Actual Draw	270	0	119
Actual Away Win	292	0	281

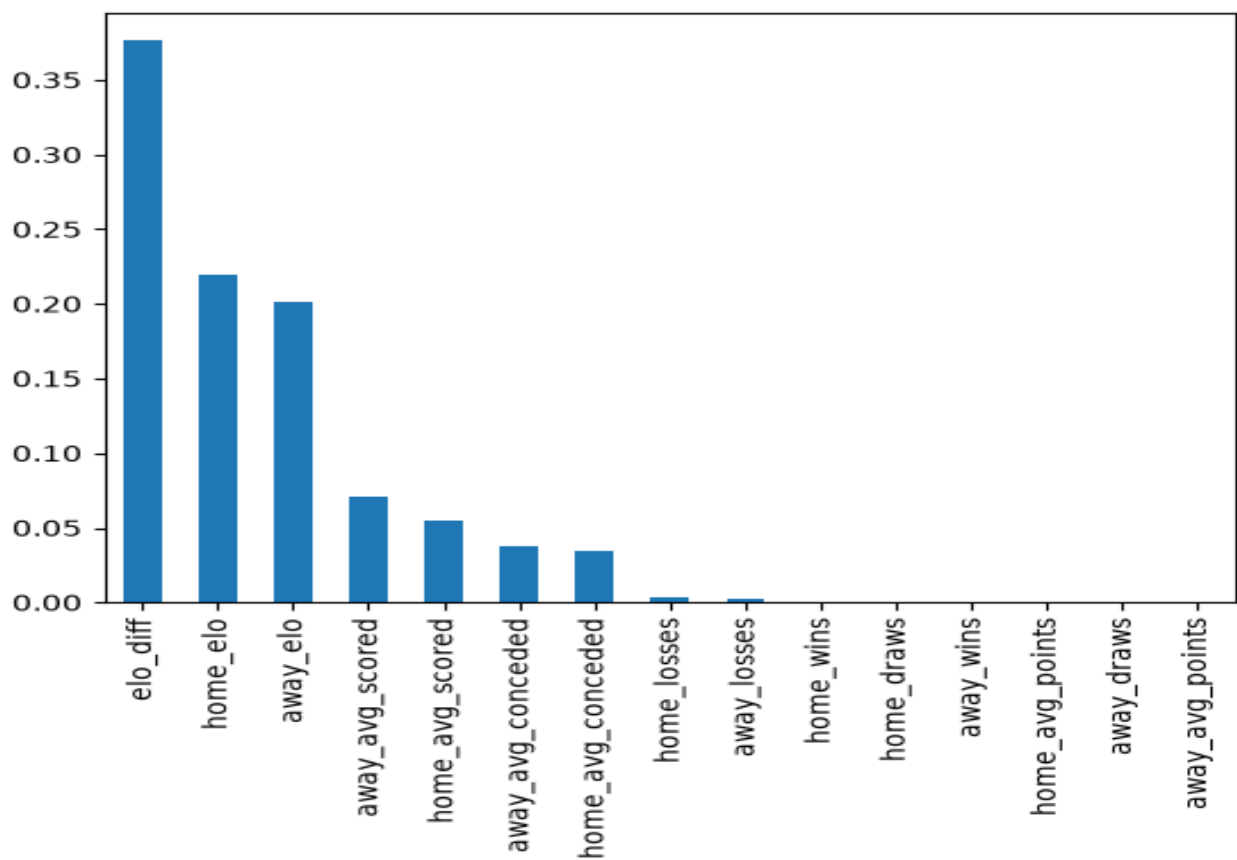
Table 6.3: Random Forest with Elo confusion matrix

c) Precision-Recall-F1 score:

	Precision	Recall	F1-Score
Home Win	0.54	0.86	0.66
Draw	0.00	0.00	0.00
Away Win	0.56	0.49	0.52

Table 6.4: Random Forest with Elo class-wise metrics

d) Importance of features:



II. Regression:

a) Home Goal MAE: 0.9842234954726656

b) Away Goal MAE: 0.9010421789483954

XGBoost:

I. Classification:

a) Accuracy Score: 0.5457725947521865

b)Confusion Matrix:

	Predicted Home Win	Predicted Draw	Predicted Away Win
Actual Home Win	648	2	103
Actual Draw	266	3	120
Actual Away Win	278	10	285

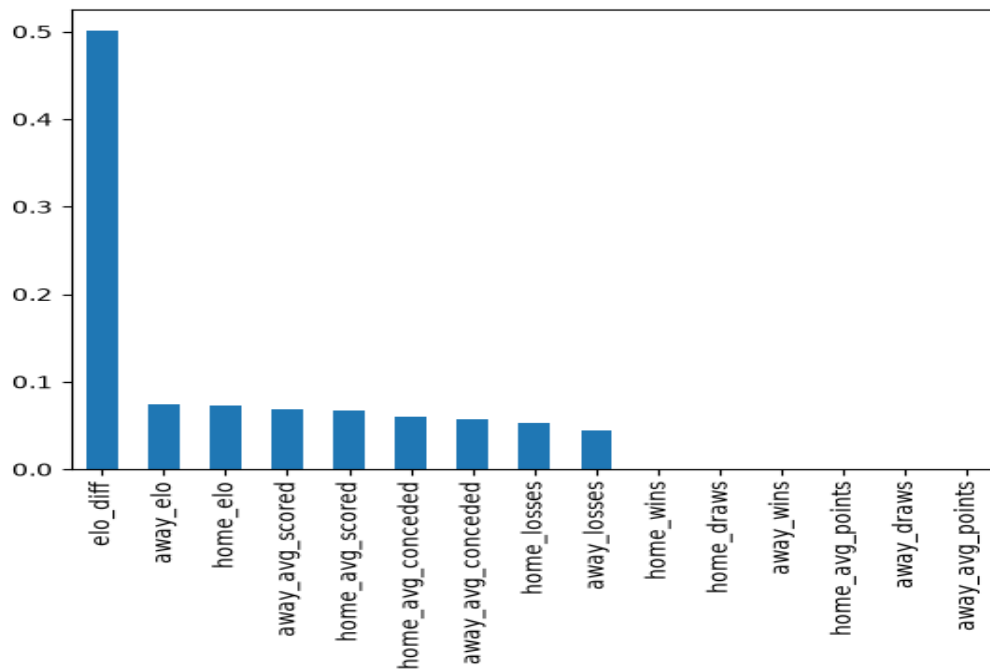
Table 6.5: XGBoost confusion matrix

c)Precision-Recall-F1 score:

	Precision	Recall	F1-Score
Home Win	0.54	0.86	0.67
Draw	0.20	0.01	0.01
Away Win	0.56	0.50	0.53

Table 6.6: XGBoost class-wise metrics

d)Importance of features:



II. Regression:

a) Home Goal MAE: 0.9780781865119934

b) Away Goal MAE: 0.9005072712898254

Neural Networks (MLP):

I. Classification:

a) Accuracy Score: 0.5422740524781341

b) Confusion Matrix:

Table 6.7: MLP confusion matrix

	Predicted Home Win	Predicted Draw	Predicted Away Win
Actual Home Win	642	0	111
Actual Draw	263	0	126
Actual Away Win	285	0	288

c) Precision-Recall-F1 score:

	Precision	Recall	F1-Score
Home Win	0.54	0.85	0.66
Draw	0.00	0.00	0.00
Away Win	0.55	0.50	0.52

Table 6.8: MLP class-wise metrics

II. Regression:

a) **Home Goal MAE:** 1.2083973858123749

b) **Away Goal MAE:** 0.903901459131072

6.6 Data Serialization and Architecture Interpretation

A critical analytical interpretation of this project concerns how trained knowledge moves from the training environment to the web interface. Running model-training loops inside the Streamlit frontend would repeat expensive work after widget interactions. The project therefore serializes trained models and state dictionaries such as `team_history` and `elo_teams` with `joblib`. The Streamlit application loads these trusted artifacts through `@st.cache_resource`, allowing subsequent prediction requests to reuse in-memory resources without retraining [9, 10].

6.7 Functional Observation Matrix

To verify that the frontend correctly interprets and displays the backend data structures, an observation matrix was utilized during functional testing.

Observation ID	Input Condition	Expected System Processing
----------------	-----------------	----------------------------

OBS-01	User selects Home Team & Away Team	The system checks the team_history dictionary for both teams' last 5 matches.
OBS-02	User selects "Basic Random Forest"	System routes the features to the basic RF classification and regression models.
OBS-03	User initiates Prediction Request	The predict_match_st function outputs discrete probabilities array and expected goals.
OBS-04	Goal Prediction Rendering	Raw regression outputs (e.g., 1.84 goals) are rounded using <code>max(0, round(home_goals))</code> .
OBS-05	Elo Page Navigation	UI maps elo_teams dictionary to GitHub logo repository URLs.

Table 6.9: Functional observation matrix

CHAPTER 7: SYSTEM DESIGN

7.1 Overall Architecture

To keep the web application responsive and avoid execution bottlenecks, EPL MatchVision uses a decoupled supervised machine-learning architecture. Placing computationally heavy model training inside an interactive Streamlit application can cause substantial delays because the script re-runs after widget interactions. The design addresses this issue by dividing the system into three isolated layers:

Layer	Components	Responsibility
Frontend (Inference) Layer	Streamlit, Custom CSS, Web Widgets	Captures user inputs (team selection, algorithm choice) and displays rendered AI probabilities and expected scorelines.
Serialization Layer	joblib, @st.cache_resource	Deserializes pre-trained ML models and historical tracking dictionaries directly into active memory to bypass redundant retraining.
Training (Backend) Layer	Python, Jupyter Notebooks, Scikit-Learn, XGBoost, Multi-Layer Perceptron(MLP)	Ingests the raw CSV dataset, calculates rolling states, optimizes hyperparameters, and exports the final binary models.

EPL Predictor (EPL MatchVision) - Overall Architecture

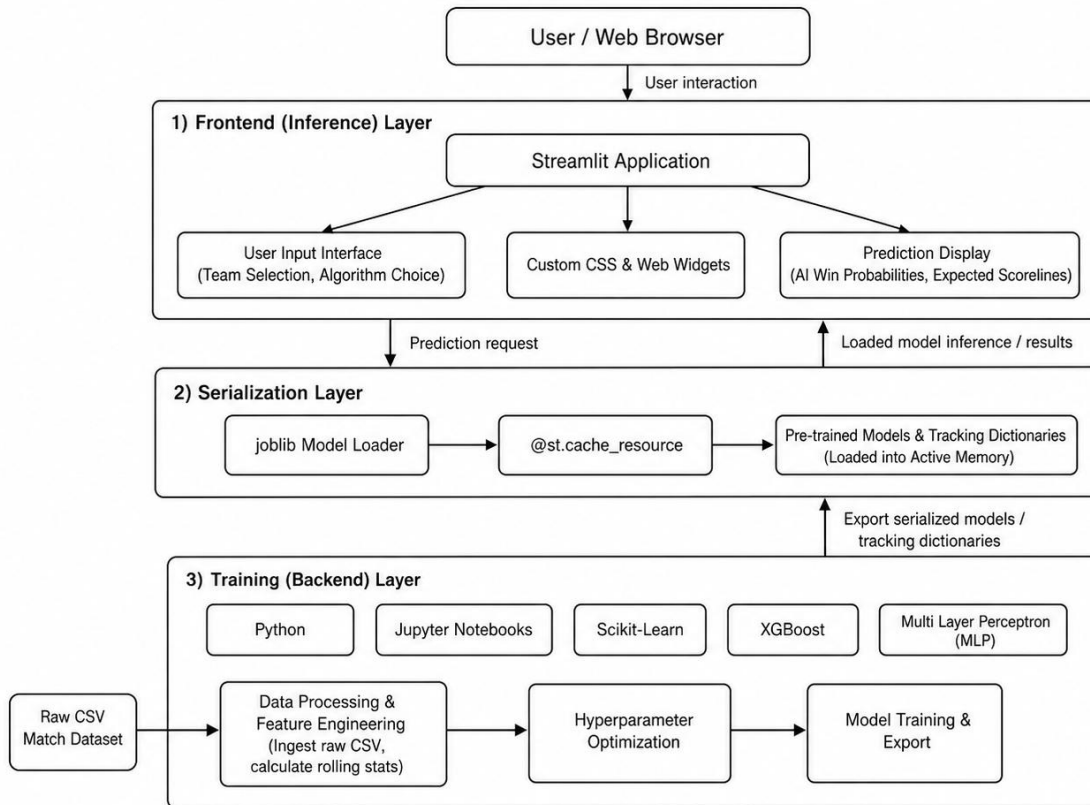


Figure 7.1: Overall Architecture of EPL Predictor (EPL MatchVision)

7.2 Data Flow Diagram - Level 0

The Level 0 context diagram models the EPL Predictor as one complete predictive process. The user supplies the selection of a Home Team, an Away Team, and a specific prediction algorithm. The system processes these inputs by exchanging them with the pre-cached .joblib models and state dictionaries, and ultimately returns the mathematical rendering of match outcomes (Win/Draw/Loss probabilities) and expected goals back to the user.

EPL Predictor (EPL MatchVision) - Level 0 Context Diagram

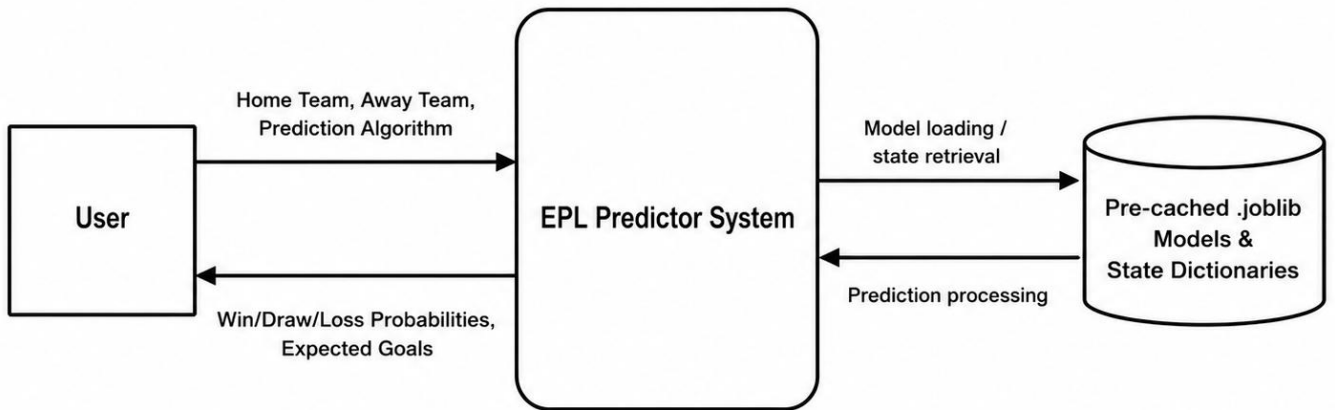


Figure 7.2: *Level 0 Context Diagram of EPL Predictor (EPL MatchVision)*

7.3 Data Flow Diagram - Level 1

The Level 1 DFD decomposes the complete process into specific functional stages:

1. **Input Capture:** The frontend receives the selected teams from dropdown menus.
2. **State Retrieval:** The system retrieves the team_history and custom elo_teams data structures from the cache.
3. **Feature Engineering:** A utility function computes the 5-match rolling averages (goals scored, goals conceded, average points) for both chosen teams.
4. **AI Inference:** The generated feature array is passed into the chosen model's predict_proba() function for discrete classification and predict() function for continuous goal regression.
5. **Formatting:** The raw regression output is rounded to realistic football integers using max(0, round(goals)).
6. **Result Display:** The frontend updates the metric containers, progress bars, and asynchronously fetched official team logos.

EPL Predictor (EPL MatchVision) - Data Flow Diagram Level 1

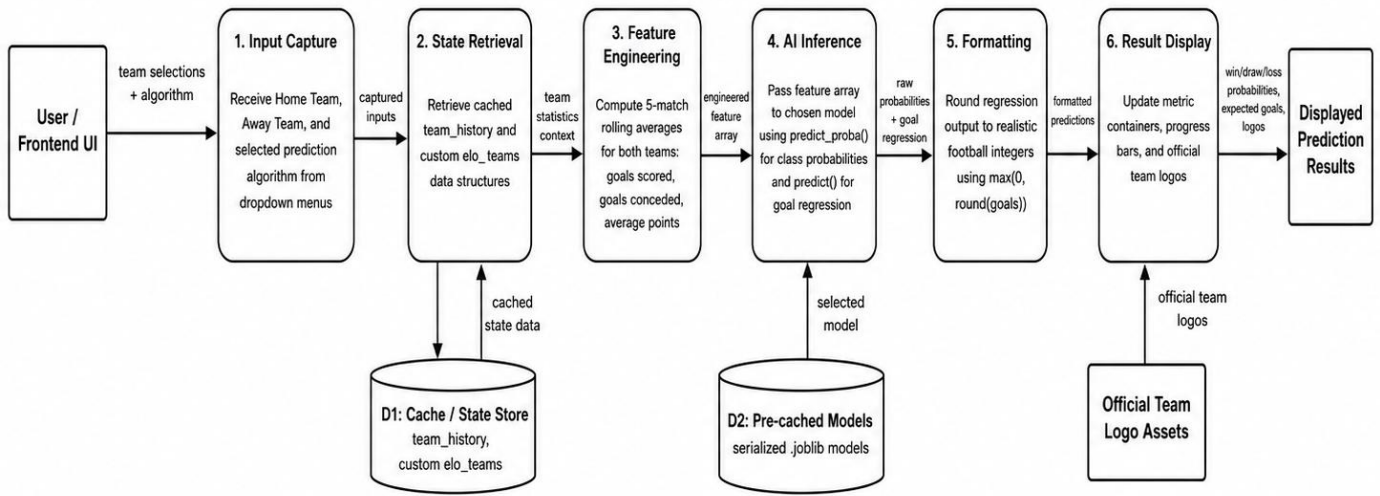


Figure 7.3: Data Flow Diagram – Level 1 of EPL Predictor (EPL MatchVision)

7.4 Use Case Model

The use-case diagram identifies the primary functions available to the user and how the system interacts with the serialized backend components.

Actor	Use Case	Description
User	Select Teams	Chooses the Home and Away clubs from a dynamically filtered list.
User	Choose Algorithm	Selects between Basic RF, RF with Elo, XGBoost, or MLP models.
User	View Predictions	Reviews the calculated win/draw/loss probabilities and expected scoreline.
User	Explore Dataset	Reviews the foundational Kaggle data (2000-2025) and model accuracy metrics.
System	Load Cached States	Bypasses training loops by loading serialized dictionaries into memory instantly.

EPL Predictor (EPL MatchVision) - Use Case Model

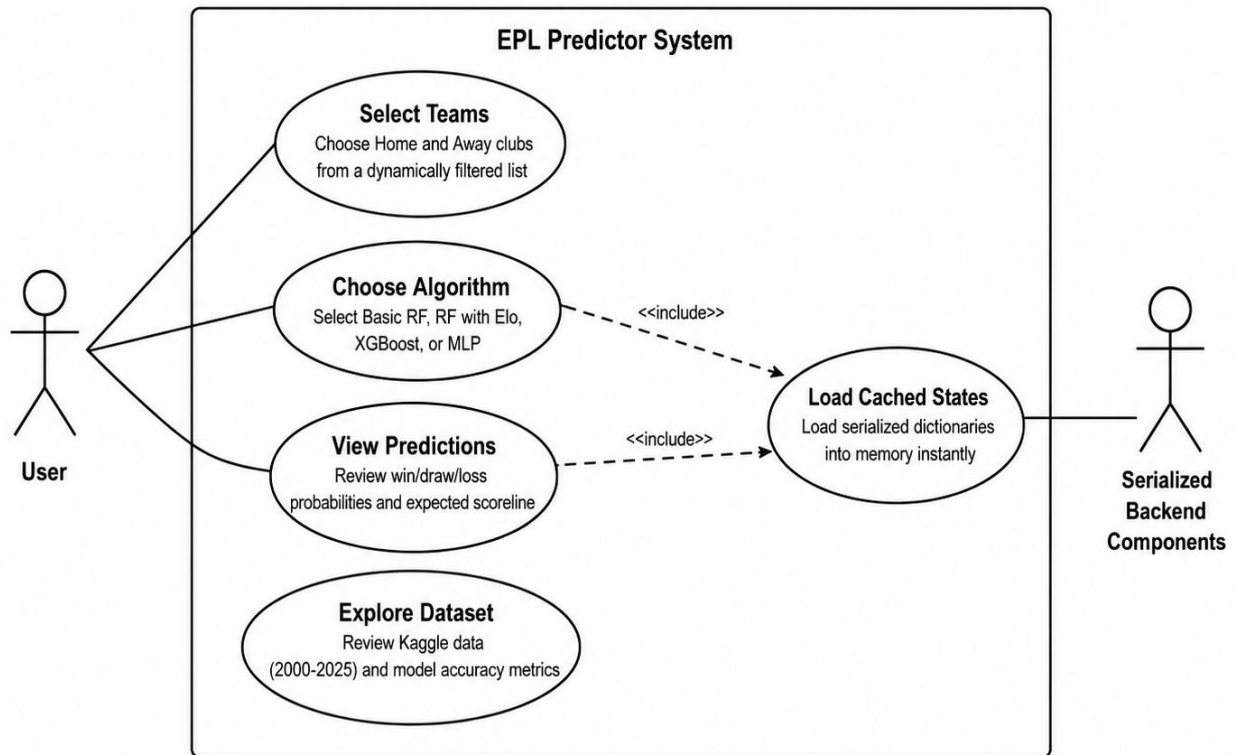


Figure 7.4: Use Case Model of EPL Predictor (EPL MatchVision)

7.5 Sequence Design

The sequence of chronological interactions for a complete prediction request is designed to reuse cached resources and avoid retraining during frontend interaction:

1. The user opens the Streamlit web application.
2. The app.py script executes the `@st.cache_resource` decorator immediately upon startup.
3. The system loads all binary .joblib models and state files into active RAM.
4. The user selects the Home Team, Away Team, and Machine Learning algorithm.
5. The `predict_match_st` function dynamically extracts features from the cached `team_history`.
6. The system executes the prediction array against the classification and regression endpoints.
7. The expected goals are mathematically rounded to realistic, discrete integer scorelines.
8. The frontend renders the visual probability progress bars and populates the dashboard.

EPL Predictor (EPL MatchVision) - Prediction Request Sequence

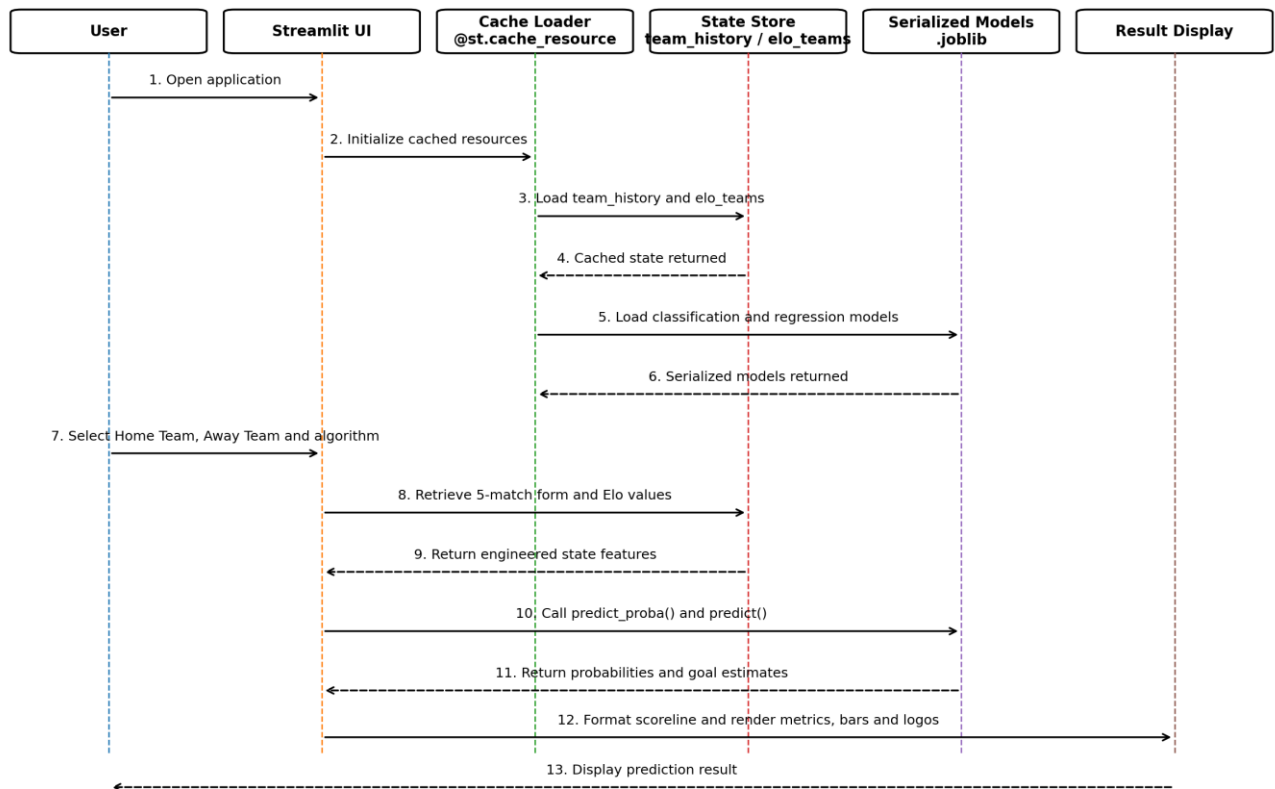


Figure 7.5: Prediction Request Sequence of EPL Predictor (EPL MatchVision)

7.6 Data Serialization & Storage Design

Unlike traditional web applications that store operational records in relational tables, this machine-learning application persists trained estimators and historical state as binary serialized objects. `joblib.dump()` is used during training and `joblib.load()` reconstructs the Python objects for inference [9]. Only trusted project-generated `.joblib` files should be loaded because `joblib` uses the pickle serialization model, which can execute code during deserialization.

EPL Predictor (EPL MatchVision) - Data Serialization & Storage Design

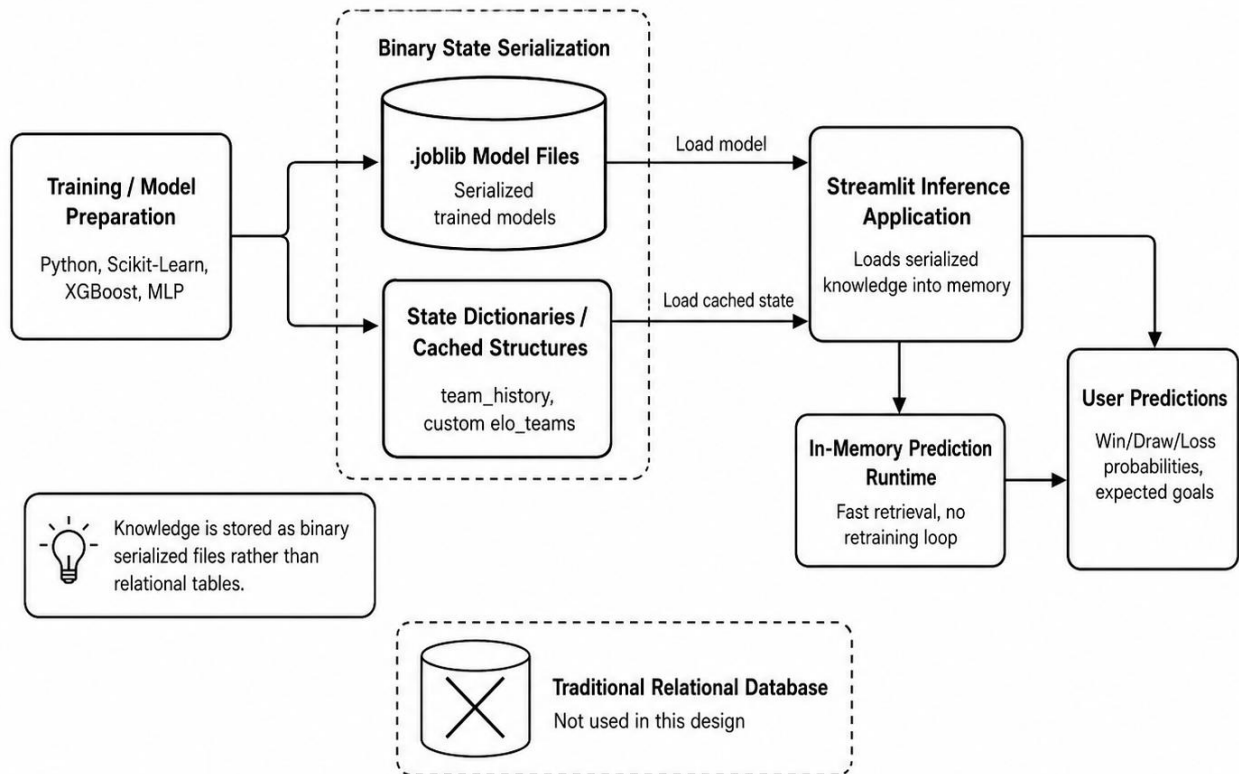


Figure 7.6: Data Serialization & Storage Design of EPL Predictor (EPL MatchVision)

CHAPTER 8: TESTING, DISCUSSION AND RESULTS

8.1 Model Performance and Accuracy Evaluation

The project successfully implemented and evaluated four distinct machine learning pipelines: Basic Random Forest (m1_basic_rf), Random Forest integrated with Elo Ratings (m2_rf_elo), XGBoost Classifier/Regressor (m3_XGBoost), and a Multi-Layer Perceptron Neural Network (m4_nn). The system was rigorously tested on two simultaneous predictive tasks:

- **Classification (Match Outcomes):** The discrete match targets (Home Win, Draw, Away Win) were evaluated using `accuracy_score`, `confusion_matrix`, and `classification_report`. To present these results to the user, the final accuracy scores for each model were serialized into binary files (e.g., `basic_rf_acc.joblib`, `xgb_acc.joblib`, `nn_acc.joblib`).
- **Regression (Expected Goals):** To predict the continuous variable of full-time goals, regression models were optimized using `RandomizedSearchCV` paired with `TimeSeriesSplit` cross-validation targeting negative mean absolute error scoring. During final testing, the goal predictions were verified using standard Mean Absolute Error (MAE) to measure by how many goals the predictions were off, and these error metrics were similarly serialized for dashboard display.

8.1.1 Comparative Summary

Table 7.1 consolidates the classification and regression results from the chronological test set. Accuracy measures the proportion of correctly classified Home Win, Draw and Away Win outcomes, while MAE reports the average absolute goal-prediction error; lower MAE is better [8].

Model	Accuracy	Home MAE	Away MAE	Mean MAE	Draw Recall	Key Finding
Basic Random Forest	48.98%	1.033	0.920	0.977	0.00	Short-term rolling-form baseline
Random Forest + Elo	54.17%	0.984	0.901	0.943	0.00	Elo adds 5.19 percentage points
XGBoost + Elo	54.58%	0.978	0.901	0.939	0.01	Best accuracy and lowest mean MAE
MLP + Elo	54.23%	1.208	0.904	1.056	0.00	No advantage over tree ensembles

Table: 8.1: Comparative model evaluation summary

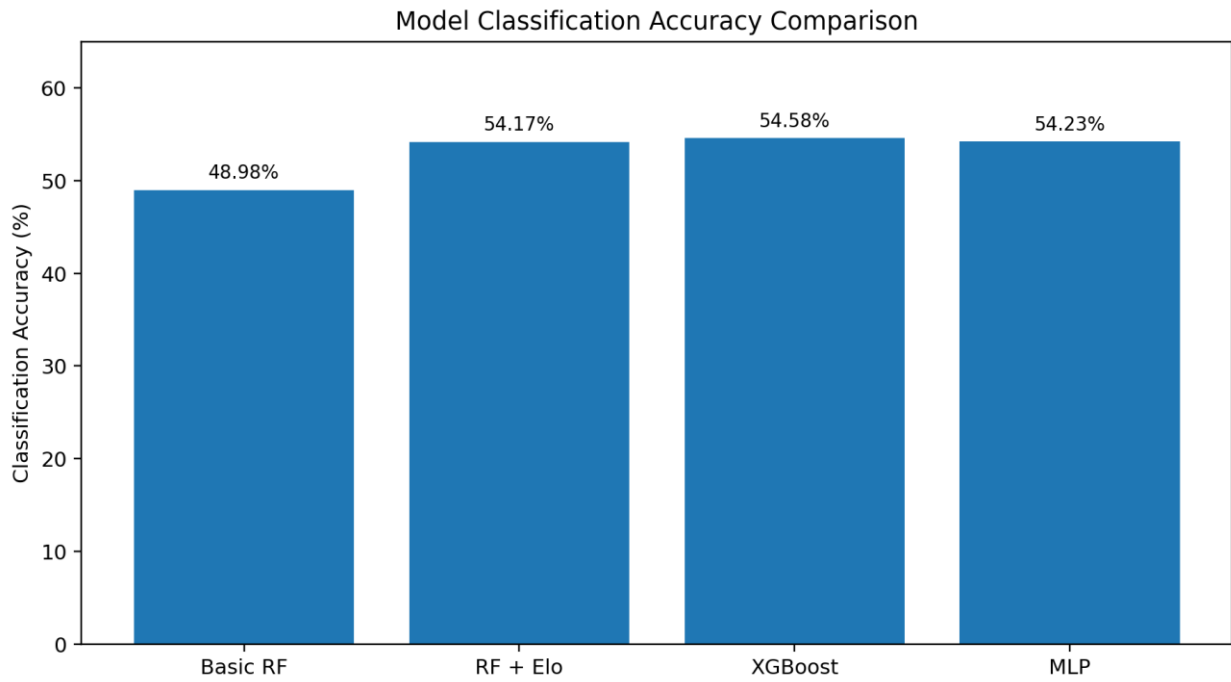


Figure 8.1: Classification accuracy comparison

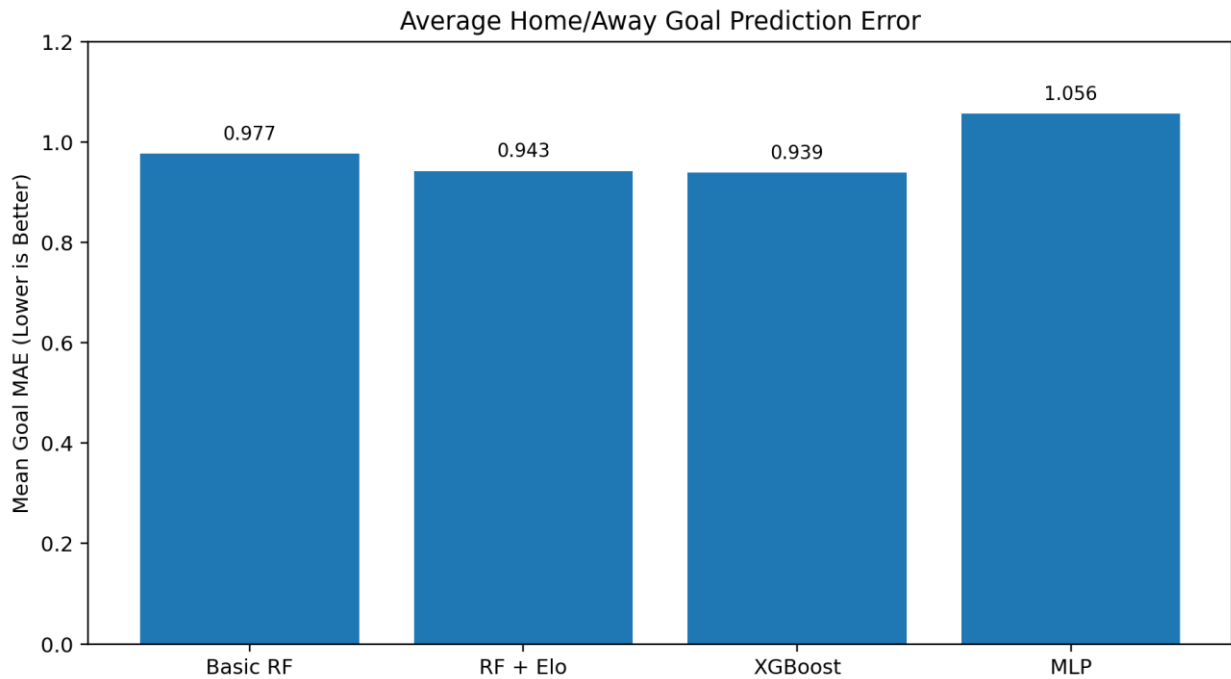


Figure 8.2: Average home/away goal MAE comparison

8.1.2 Effect of the Elo Feature

Adding Elo ratings to Random Forest increased accuracy from 48.98% to 54.17%, an absolute improvement of 5.19 percentage points (10.60% relative). Mean goal MAE decreased from 0.977 to 0.943 goals, an improvement of approximately 3.48%. This is the clearest quantitative evidence that long-term team-strength information improves the project baseline. XGBoost produced only a further 0.41 percentage-point gain over Elo Random Forest, suggesting that the Elo feature contributed more than the change between the two tree-ensemble algorithms.

8.1.3 Class-Wise Error Analysis

The confusion matrices reveal a critical limitation: Basic Random Forest, Elo Random Forest and MLP did not correctly identify any draw in the test set, while XGBoost identified only 3 of 389 draws (recall 0.01).

Therefore, overall accuracy alone is insufficient. The best model, XGBoost, reached 54.58% accuracy, which is 10.67 percentage points above the majority-class baseline of 43.91%, but draw recognition remains weak. Future evaluation should prioritize macro-F1, balanced accuracy, probability calibration and class-imbalance treatment.

8.2 Application Performance and Serialization Results

A major technical result of this project is the successful resolution of Streamlit frontend execution bottlenecks. Initially, integrating machine learning models directly into web scripts caused the entire script to re-run top-to-bottom upon any user interaction, forcing massive execution delays.

The implemented solution uses a decoupled architecture: training is isolated in Jupyter Notebooks, while trained algorithms and dynamic tracking dependencies are stored in the `saved_models` directory through `joblib`. Streamlit loads these resources once through `@st.cache_resource` [9, 10]. Functional testing confirmed that subsequent user interactions execute inference without starting a new training loop. A controlled hardware-independent latency benchmark was not recorded; therefore, this report claims elimination of redundant retraining rather than a universal sub-second timing guarantee.

8.3 Interface and Visualization Achievements

The operational outcome of the project is a highly responsive web application that successfully overrides the generic Streamlit appearance to offer a premium developer IDE aesthetic mimicking Google AI Studio. Key visual results include:

- **Custom Theming:** The deployment of an "Under the Floodlights" aesthetic, featuring a deep pitch-black/green foundation, frosted-glass sidebar panels (`backdrop-filter: blur`), and sharp neon typography (utilizing fonts like *JetBrains Mono* and *Oswald*).
- **Dynamic Data Rendering:** The raw classification probabilities extracted via the `predict_proba()` function are seamlessly mapped to interactive visual progress bars. Furthermore, the raw regression outputs for expected goals are mathematically rounded using `max(0, round(home_goals))` to return realistic, discrete scorelines to the user.
- **Asset Integration:** To provide a polished sports dashboard experience, team logos were successfully integrated.

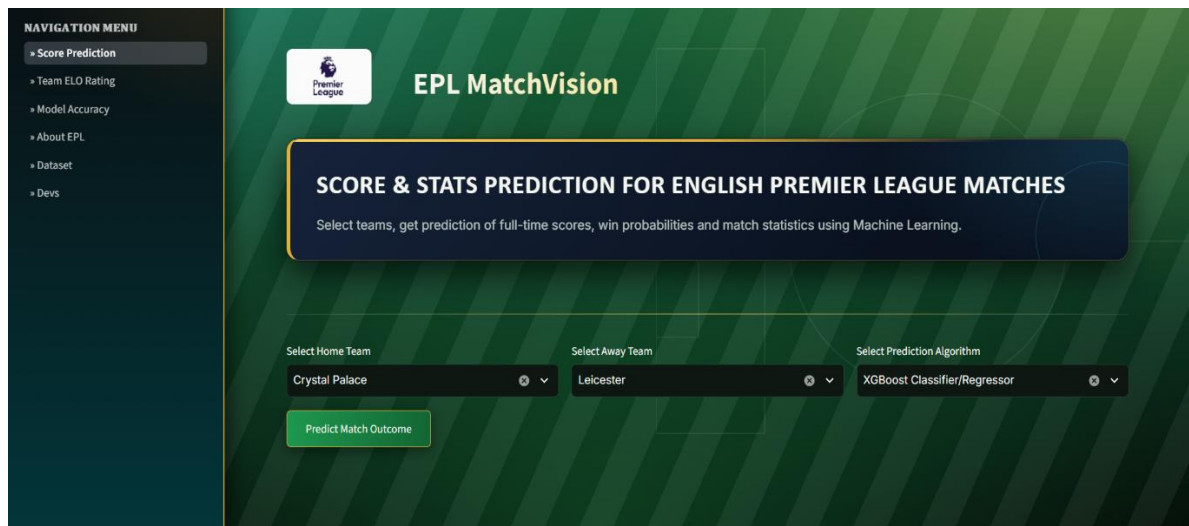


Figure 8.3: Home Page with Navigation Bar



Figure 8.4: Model performance dashboard showing comparative accuracy metrics

Logo	EPL Team	ELO Rating
	Arsenal	1758.75
	Aston Villa	1760.04
	Birmingham	1478.78
	Blackburn	1425.72
	Blackpool	1473.34
	Bolton	1475.10
	Bournemouth	1617.78
	Bradford	1374.01
	Brentford	1633.60
	Brighton	1623.21

Figure 8.5: Team Elo ratings dashboard

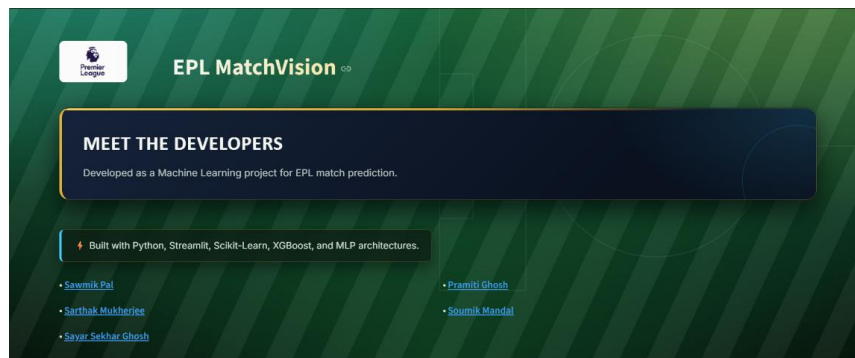


Figure 8.6: Developer information

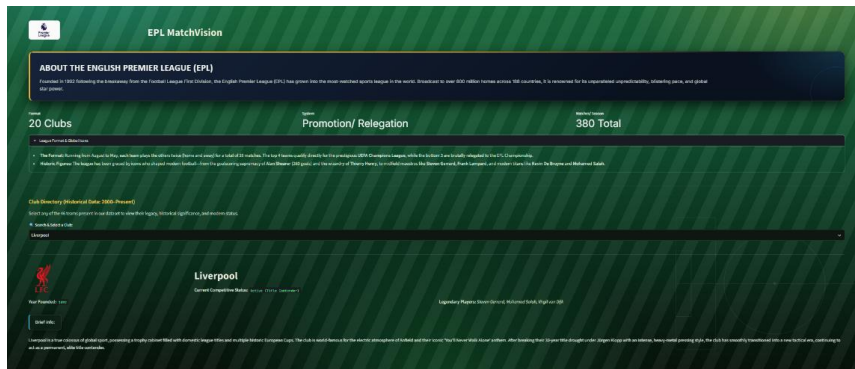


Figure 8.7: Information About EPL & Teams

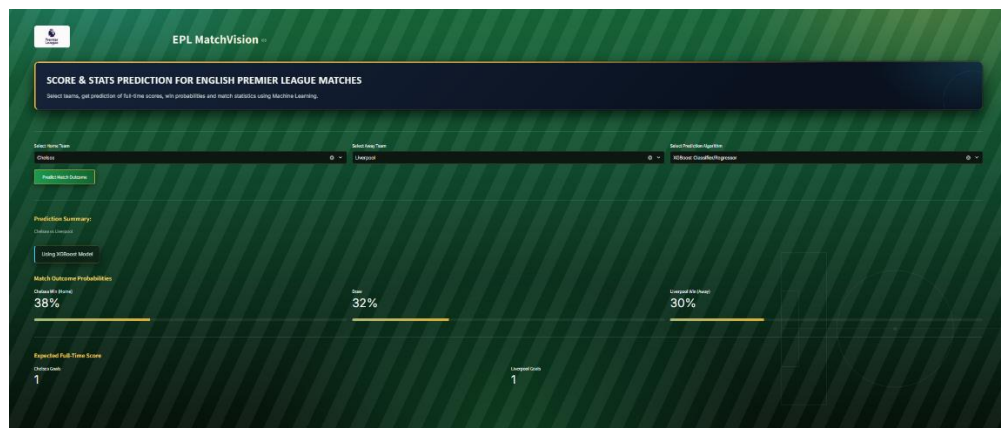


Figure 8.8: Prediction dashboard showing probabilities and expected scoreline

8.4 Ultimate Project Outcome and Objective Fulfilment

The final deliverable is a working multi-page Streamlit decision-support application that converts historical EPL records into selectable model predictions. It provides team selection, algorithm selection, win/draw/loss probabilities, expected scorelines, a raw-data explorer, Elo tracking, model-performance views and official team-logo rendering. Table 7.2 links the stated objectives to verified project outputs.

Objective	Achieved Result	Evidence
Interactive prediction interface	Home/Away selectors, algorithm selector and formatted prediction output	Prediction dashboard (Figure 7.8)
Multi-model comparison	Basic RF, RF + Elo, XGBoost and MLP evaluated on a chronological test set	Table 7.1 and Figures 7.1-7.2
Short- and long-term feature engineering	Five-match rolling statistics combined with custom Elo ratings	Section 5.3 and elo_all.joblib
Efficient deployment	Training separated from inference; models and state loaded from joblib cache	Figures 6.1 and 6.6
Analytical transparency	Model Performance, Elo and Information About EPL & Teams pages included in the application	Figures 7.4,7.5,7.7

Table: 8.2: *Objective fulfilment and final project outcomes*

8.5 Discussion and Strengths of the Project

The analytical findings confirm that predicting football match outcomes is an excellent application of Supervised Machine Learning. By engineering features from labeled historical data, the models successfully learned mathematical mappings to predict results without relying on the trial-and-error approach required by reinforcement learning.

A core analytical strength discovered during testing was the value of the custom Elo rating system (elo_all.joblib). While the Basic Random Forest model relied purely on short-term form—calculating a rolling average of goals and points over a team's last five matches—the introduction of Elo ratings provided

the advanced XGBoost and MLP models with a much more robust understanding of a team's long-term historical dominance across multiple seasons. Additionally, expanding the application to include dedicated routing modules like `Elo_pg.py` (Team Elo Ratings), `model_acc_pg.py` (Model Performance), and `dataset.py` transforms the tool from a simple script into a comprehensive analytical ecosystem.

8.6 Limitations and Threats to Validity

While the system architecture is highly optimized, it is bound by certain limitations. The predictive models rely strictly on structured historical data (rolling stats and custom Elo ratings); they cannot account for real-world, unquantifiable disruptions such as unexpected player injuries, weather interference, or sudden managerial changes. Furthermore, because the models and state dictionaries are pre-trained and serialized as `.joblib` files, the system cannot automatically ingest live, real-time match statistics without an administrator manually updating the Kaggle dataset and re-running the backend training note books.

CHAPTER 9: CONCLUSION

9.1 Project Summary

The **EPL Predictor** project has successfully culminated in the development of a high-performance web application designed to forecast English Premier League match results. By utilising a comprehensive dataset spanning from **2000 to 2025**, the system provides users with data-driven insights into match outcomes and specific goal counts. The project highlights the transition from complex **Supervised Learning** training in Jupyter Notebooks to a streamlined, real-time prediction environment using **Streamlit**.

9.2 Fulfilment of Objectives

The team—**Sawmik Pal, Sarthak Mukherjee, Sayar Sekhar Ghosh, Pramiti Ghosh, and Soumik Mandal**—successfully achieved all primary project objectives:

- **Outcome Forecasting:** The system generates probability estimates for Home Win, Draw and Away Win. XGBoost achieved the highest observed test accuracy of 54.58%; this represents moderate predictive performance, not certainty.
- **Score Prediction:** The regression pipelines estimate full-time home and away goals. XGBoost produced the lowest average goal MAE of 0.939, meaning predictions differed from observed goals by approximately 0.94 goals on average across the two targets.
- **Dynamic Rating:** The custom Elo system provides a long-term team-strength signal. Random Forest with Elo improved accuracy by 5.19 percentage points over the rolling-form-only baseline and reduced average goal MAE by about 3.48%.

9.3 Technical and Design Achievements

- **Serialization and Efficiency:** Joblib serialization and Streamlit resource caching allow the application to reuse trained models and state dictionaries without retraining them during normal user interactions.
- **User Experience:** The "MatchVision" theme, featuring a "broadcast lower-third" aesthetic and glassmorphism design, successfully transforms complex statistical data into an intuitive and visually

engaging interface.

- **Modular Architecture:** The separation of concerns between the frontend (Streamlit), utility logic (feature engineering), and the pre-trained model binaries allows for easy maintenance and future scalability.

9.4 Final Remarks

In conclusion, EPL MatchVision is a robust proof-of-concept for supervised learning in sports analytics. The project demonstrates that combining short-term rolling form with long-term Elo ratings improves the baseline, while XGBoost provides the strongest overall result among the evaluated models. The weak draw recall also shows that the application should be used as an analytical aid rather than a definitive forecasting engine. The system architecture, serialized inference pipeline and transparent model-comparison interface provide a sound basis for further research and deployment improvements.

CHAPTER 10: RECOMMENDATIONS AND FUTURE SCOPE

10.1 Address Class Imbalance

Apply class weighting, stratified temporal sampling where valid, threshold tuning, or resampling restricted to the training period. Because draws are the weakest class, macro-F1 and balanced accuracy should be optimized alongside overall accuracy.

10.2 Improve Probability Quality

Evaluate log loss, Brier score and reliability curves. Probability calibration using Platt scaling or isotonic regression should be fitted only on chronologically valid validation data.

10.3 Extend Football Features

Add player availability, starting line-ups, rest days, injuries, home advantage, expected-goals history, managerial changes and fixture congestion. These features may improve draw and close-match discrimination.

10.4 Automate Data Refresh and Retraining

Create a scheduled data-ingestion pipeline, validate new records, retrain models after each season or matchweek, and version datasets, features and serialized model artifacts.

10.5 Benchmark Deployment Performance

Measure cold-start time, cached model-load time, per-prediction latency, memory use and concurrent-user behaviour on defined hardware. Report medians and percentiles rather than informal timing observations.

10.6 Strengthen Reproducibility and Security

Store package versions, random seeds, hyperparameters, data-split dates and model hashes. Load joblib artifacts only from trusted sources and consider safer portable formats when deployment requirements permit.

REFERENCES

- [1] M. Hui, “English Premier League (EPL) Match Data 2000–2025,” Kaggle, 2025. [Online]. Available: <https://www.kaggle.com/datasets/marcohuui/english-premier-league-epl-match-data-2000-2025>. [Accessed: 14-Jul-2026].
- [2] A. E. Elo, The Rating of Chessplayers, Past and Present, 2nd ed. New York, NY, USA: Arco Publishing, 1978.
- [3] Scikit-learn Developers, “RandomForestClassifier and RandomForestRegressor,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#forest>.
- [4] XGBoost Developers, “Python API Reference and Scikit-Learn Estimator Interface,” XGBoost documentation. [Online]. Available: <https://xgboost.readthedocs.io/en/stable/python/>.
- [5] Scikit-learn Developers, “MLPClassifier and MLPRegressor,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html.
- [6] Scikit-learn Developers, “TimeSeriesSplit,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html.
- [7] Scikit-learn Developers, “RandomizedSearchCV,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html.
- [8] Scikit-learn Developers, “Model evaluation: accuracy_score, confusion_matrix and mean_absolute_error,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html.
- [9] Joblib Developers, “Persistence: joblib.dump and joblib.load,” joblib documentation. [Online]. Available: <https://joblib.readthedocs.io/en/stable/persistence.html>.
- [10] Streamlit, “st.cache_resource,” Streamlit documentation. [Online]. Available: https://docs.streamlit.io/develop/api-reference/caching-and-state/st.cache_resource.
- [11] Streamlit, “Caching overview and application architecture,” Streamlit documentation. [Online]. Available: <https://docs.streamlit.io/develop/concepts/architecture/caching>.
- [12] L. Szczecinski and A. Djebbi, “Understanding and Pushing the Limits of the Elo Rating Algorithm,” arXiv:1910.06081, 2019.

APPENDICES / ANNEXURES

APPENDIX A: EVALUATION NOTES

Table A.1 consolidates the essential interpretation notes that should accompany the model results.

Item	Evaluation Note
Test-set composition	1,715 chronological matches: 753 Home Wins, 389 Draws and 573 Away Wins.
Accuracy interpretation	Overall accuracy must be read with class-wise precision, recall and F1-score because the classes are imbalanced.
MAE interpretation	The XGBoost mean home/away MAE of 0.939 represents an average absolute goal error of approximately 0.94.
Known limitation	Draw recall is extremely low; the dashboard therefore displays probabilities and should not be treated as certain.

Table A.1: Consolidated evaluation notes

APPENDIX B: MODEL AND STATE ARTEFACT INVENTORY

Table B.1 identifies the principal serialized resources used by the inference application.

Artefact	Purpose
Basic RF model files	Rolling-form classification and goal-regression baseline.
RF with Elo model files	Random Forest inference using rolling statistics and Elo features.
XGBoost model files	Boosted-tree classification and regression endpoints.
MLP model files	Neural-network classification and goal-regression endpoints.
team_history.joblib	Cached recent-match records used to calculate five-match form.
elo_all.joblib	Final team-rating dictionary used for long-term strength features.

Table B.1: Serialized model and state artefact inventory

ANNEXURE I: FINALIZATION AND PRINTING COMPLIANCE

Before final printing, the students must obtain clearance from the project guide. The final copy should be printed on A4 paper, use Times New Roman body text at 12 pt with 1.5-line spacing, maintain 1.5-inch left and 1-inch right/top/bottom margins, place page numbers at the bottom, verify that every table and figure is cited in the text, and be hard bound with suitable cover embossing.

APPENDIX C: DATA DICTIONARY AND ENGINEERED FEATURES

The predictive pipeline uses raw match fields together with engineered recent-form and long-term-strength variables. Table C.1 documents the principal fields so that the data preparation process can be reproduced and audited. All rolling and Elo values are calculated chronologically, using only information available before the match being predicted.

Field / Feature	Meaning	Role in the Model
FullTimeResult	Encoded final outcome: Home Win, Draw or Away Win.	Classification target.
FTHG / FTAG	Full-time home and away goals.	Regression targets and historical inputs.
GoalDiff	Home goals minus away goals for a historical match.	Represents scoring dominance.
ShotDiff	Difference between home and away total shots.	Captures attacking volume.
ShotOnTargetDiff	Difference in shots on target.	Captures shot quality and pressure.
home_avg_scored	Home team mean goals scored over its last five matches.	Short-term attacking form.
away_avg_scored	Away team mean goals scored over its last five matches.	Short-term attacking form.
home_avg_conceded	Home team mean goals conceded over its last five matches.	Short-term defensive form.
away_avg_conceded	Away team mean goals conceded over its last five matches.	Short-term defensive form.
home_avg_points / away_avg_points	Average league points obtained in the last five matches.	Recent overall performance.
home_elo / away_elo	Team Elo ratings immediately before the fixture.	Long-term relative strength.
elo_diff	Home Elo minus away Elo.	Direct strength-gap feature.

Table C.1: Data dictionary for raw and engineered predictive features

Data-quality checks remove or repair invalid dates, inconsistent team names and missing numerical values before feature calculation. The chronological ordering is then locked so that future matches cannot influence earlier feature values. This control is essential for preventing data leakage and for making the reported test accuracy meaningful.

APPENDIX D: EVALUATION FORMULAE AND INTERPRETATION

The classification and regression results are evaluated with complementary metrics because no single number fully describes model behaviour. The following formulae define the measures used in the report.

Accuracy: $\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$. Measures the overall proportion of correctly classified outcomes.

Precision: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$. Shows how often predictions for a class are correct.

Recall: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$. Shows how many actual examples of a class are detected.

F1-Score: $\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. Balances precision and recall for each outcome class.

Mean Absolute Error: $\text{MAE} = (1/n) \sum |\text{Actual Goals} - \text{Predicted Goals}|$. Expresses goal-regression error directly in goal units.

Elo Expected Score: $E_A = 1 / (1 + 10^{((R_B - R_A)/400)})$. Estimates Team A's expected result from the rating difference.

Elo Rating Update: $R'_A = R_A + K(S_A - E_A)$. Updates team strength after the observed result.

Metric	Preferred Direction	Interpretation in This Project
Accuracy	Higher	XGBoost obtained the highest observed value of 54.58%.
Class Recall	Higher	Draw recall exposes the principal classification weakness.
F1-Score	Higher	Used to compare class performance when precision and recall differ.

Metric	Preferred Direction	Interpretation in This Project
MAE	Lower	XGBoost obtained the lowest mean home/away error of 0.939 goals.

Table D.1: Interpretation direction for the principal evaluation metrics

The Elo-enhanced comparison is interpreted as an ablation-style result: the Random Forest accuracy increased from 48.98% to 54.17% when long-term team-strength variables were added. This does not prove that Elo alone causes every improvement, but it provides direct evidence that the additional strength representation is useful within the implemented pipeline.

APPENDIX E: FUNCTIONAL TEST CASES

Table E.1 records representative end-to-end tests used to verify that the Streamlit interface, cached state, model routing and result formatting work together correctly. The status indicates the observed result in the completed prototype.

Test ID	Test Condition	Expected Behaviour	Status
TC-01	Open the application	Home page and navigation controls load without starting model training.	Passed
TC-02	Select two different EPL teams	Both selections are accepted and the prediction action is enabled.	Passed
TC-03	Select the same team twice	The interface prevents or filters an invalid self-fixture.	Passed
TC-04	Choose Basic Random Forest	The request is routed to the baseline classifier and regressors.	Passed
TC-05	Choose RF with Elo, XGBoost or MLP	The matching serialized model pair is loaded and used.	Passed
TC-06	Request a prediction	Win/Draw/Loss probabilities and expected goals are returned.	Passed
TC-07	Receive negative or fractional raw goal output	Formatting applies $\max(0, \text{round}(\text{goals}))$ before display.	Passed
TC-08	Open Dataset, Elo or Model Performance page	The selected secondary page displays the associated records or metrics.	Passed

Table E.1: Representative functional and integration test cases

The tests confirm functional integration rather than statistical certainty. Model quality is assessed separately through the chronological test set, confusion matrices, class-wise reports and goal MAE. Repeating these tests after any dataset refresh or model replacement provides a practical regression-testing checklist for future maintenance.

ANNEXURE II: INSTALLATION AND EXECUTION GUIDE

This annexure provides a concise procedure for reproducing the deployed application from the project files. The exact package versions used for the final submission should be retained in requirements.txt or an equivalent environment record.

N.1 Prerequisites

A computer with Python, pip, the project source directory and the trusted project-generated .joblib artefacts is required. The raw dataset and Jupyter notebooks are needed only for retraining; the Streamlit inference application can operate from the saved models and state dictionaries.

Command / Action	Purpose
<code>python -m venv .venv</code>	Create an isolated Python environment.
Activate the virtual environment	Ensure project packages are installed locally.
<code>pip install -r requirements.txt</code>	Install Streamlit, pandas, NumPy, scikit-learn, XGBoost and joblib dependencies.
Verify saved_models/	Confirm that model and state .joblib files are present and trusted.
<code>streamlit run app.py</code>	Launch the EPL MatchVision web interface.

Table N.1: Installation and application-launch steps

N.2 User Workflow

After the browser opens, the user selects the Home Team, Away Team and preferred algorithm. The application retrieves five-match form and Elo state, constructs the feature array, performs classification and regression inference, and displays the probability bars and expected scoreline. The Dataset, Elo Ratings and Model Performance pages can then be used to inspect the supporting data and evaluation results.

N.3 Maintenance and Final Submission

When new match data is added, the team should validate the schema, rerun the chronological feature-engineering and training notebooks, record the package versions and random seeds, and replace the serialized artefacts as one versioned set. Before hard binding, the guide should approve the final content, page numbers, contents table, illustration references and printed layout.
reported metrics.